

Retrospective Report

1 Timeline and Reflection

1.1 Low fidelity Prototype

Expected Completion: Week 3, **Actual Completion:** Week 5

Goal: Create a low fidelity prototype that outlines the functions of the program.

Progress: Complete.

As mentioned within the introduction, I was able to use figma to create a prototype of what I wanted the Star Student app. Using a diagram of the general requirements and features, these were then translated to a version of the UI design. I attempted to translate concepts taken from my User Centred Design, ensuring that the interface is:

- Consistent
- Avoids unneeded visual clutter
- Groups similar concepts together
- Utilises features similar to common apps for learnability
- Tie in motifs from the real world
- Flexible
- Efficient

And in this prototype, it was possible to test the navigation and analyse if users would be able to navigate decently on the first try.

It had taken longer than expected to understand the complete scope of features and how I could showcase them on Figma. Despite the delay, this has been an extremely valuable piece reference for the overall UI development.

1.2 Database Connection

Expected Completion: Week 4, **Actual Completion:** Week 8

Goal: Add database read and write capabilities to the app, accessible via the UI.

Progress: Complete.

A large part of this app relies on the processing of data and representing that to the user in real time, so finding a database these features became crucial as any delays or poor integrations could greatly affect the user experience badly. System status, like loading symbols, can be shown but this can still lead to frustration.

Firebase was a platform that kept appearing in my research and had a significant amount of documentation, which was very useful due to my lack of prior knowledge of the service. This also provided a chance to learn the basics about implementing the UI and the structure required by the project.

1.3 Attempts to set up a CI/CD pipeline

Expected Completion: - Week 3, **Actual Completion:** N/A

Goal: Add a CI/CD pipeline for tests after pushing to gitlab.

Progress: Uncomplete/blocked.

Several attempts have been made to add a CI/CD that at minimum was expected to run the unit and UI tests once pushed.

Initially, I have been attempting to connect my app to the Google Play store, as it would be possible to easily share the code with testers and receive additional feedback that could be reintegrated with development. However, some permission issues continued and didn't allow for the authentication process to be completed.

Later, I had simplified and made an attempt to fix the .gitlab-ci.yml, however several runtime errors have continued to persist, leading the pipeline to fail every time.

Due to wanting to enforce good software engineering practices and being able to store errors that can then be used for future bugfixes, this will likely be readdressed later on within development.

1.4 UI Components

Expected Completion: Week 3, **Actual Completion:** Week 5

Goal: Create the UI components to be populated throughout the app's screens.

Progress: Complete.

To stick to the consistent design, I decided to use a segmented approach where UI components like backgrounds, buttons, widgets would all be called to a screen to be displayed. This creates a centralised place where changes are then globally applied and are not needed to be manually changed for every page. Many references were made to my Figma prototype for the actual look, and then variables are then implemented to customise the component for the situation.

This may have taken longer than expected, but I believe it should reduce the overall technical debt and improve the code quality. Not only does this support future development but it also makes possible user customisation easier to implement, as any changes cascade throughout the whole app.

1.5 View Screens

Expected Completion: Week 4, **Actual Completion:** Week 9

Goal: Create the basic UI screens that are shown to the user.

Progress: Complete, with further development in Term 2.

After building the components, creating the screen for each page became significantly easier, as each screen acts like a container for all the components. Due to the clean architecture approach I have been using, I will be creating all required views (screens) within a single layer for each feature. This way I can ensure the general quality of each feature has been fully tested and integrates well with any prior development.

1.6 Navigation

Expected Completion: Week 4, **Actual Completion:** Week 9

Goal: Add basic navigation through the app to the different screens.

Progress: Complete, with further development in Term 2.

Navigation is crucial so that the user is able to move around the app without significant actions, so this was a task that I attempted soon after some of the basic UI screens had been implemented. Several interfaces had been added to ensure that all components and attributes are accessible but without the potential for data leaks. These interfaces support the overall readability due to its simplified nature.

Further edits might need to be made in the future, as currently the app's navigation is fairly linear, however the segments structure should be able to support this.

1.7 Testing

Expected Completion: Week 9, **Actual Completion:** Week 11

Goal: Add unit and UI testing to the current versions of the code system.

Progress: Complete, with further development in Term 2.

With an attempt to have as high a percentage for the test coverage for my project, to ensure that the models and UI are able to work as expected and fail safely so the user doesn't lose any progress from errors that arise or get stuck at a particular point.

All of the unit tests for the model follow an action -> behaviour -> result pattern, ensuring that every branch gets addressed and has an expected outcome. These tests can then be run and checked for their overall coverage using JUnit as Kotlin is also compactable. Android studio allows for overall test coverage to show, alongside a percentage of the lines covered.

UI testing has been a little harder, as it requires the emulator to interact with user actions, which are harder to replicate. However, due to the composable nature of the UI components, testing tags can be added to each component and their features, and then asserted whether or not it has been rendered within the emulator. Also, UI test coverage requires further steps to find out, as it requires a Jacoco report to be generated which then shows all the components and functions that are hit with the tests.

Additional testing such as security and end-to-end testing will be addressed later once a stable build has been created.

1.8 Theory and Report

Expected Completion: Week 3, **Actual Completion:** Week 11-13

Goal: Complete research and report required for development.

Progress: Complete, with further development in Term 2.

The overall learning curve of android app development was significantly steeper than expected and it took a longer time to understand. Many of the concepts covered in the report and my research have significantly helped my overall design process and ideas, however the actual development has been quite difficult to implement.

The current foundation that I have managed to create should support the future development and customisable interactions that I plan to implement for the user. So this will need to be speed up over the term ahead.

2 Term Two plans

Week	Primary Deliverable	Includes	Achievement	PERT Value (Weeks)
------	---------------------	----------	-------------	--------------------

1-2 (12/01 – 25-01)	GPS location tracking and Study space implementation	Tracking the user via GPS, recording specific coordinates, recognising users within recorded 'study spaces', unit testing	Allow users to be tracked using GPS and recognise 'study spaces'	M(1.75), O(1.5), P(2) - 1.75
3 (26/01 - 01/02)	Achievements Page	Setting some predetermined conditions for each achievement, sending a notification once reached	Allow users to receive achievements and view possible options	M(1), O(0.5), P(1.5) - 1
4-5 (02/02 – 15/02)	Planner and Calendar view	Calendar view, Task view, Event view, Sync with existing Calendars, Categories, Manual entries, unit testing	Create a simple to use drag and drop calendar	M(1.75), O(1.5), P(2.25) - 1.8
6 (16/02 - 22/02)	History Page	History page, previous study session, previous diary logs	A history page is available with all previous study session, diary logs and achievements	M(1), O(0.5), P(1.5) - 1
7-8 (23/02 - 08/03)	Homepage customisation + Themes	Homepage navigations can be added or removed, homepage	The homepage and themes is able to be dynamically changed without affecting the overall system	M(1.75), O(1.5), P(2.25) - 1.8
9 (09/03 - 15/03)	Testing and debugging	All code is fully tested and debugged, Security testing, system testing, functional testing	Stable version of all working code	M(1), O(0.5), P(1.5) - 1

10-11 (16/03 - 29/03)	Prepare for final submission	Final Report, poster, Presentation	Complete final report, poster and presentation	M(1.75), O(1.5), P(2.25) - 1.8
------------------------------	------------------------------	------------------------------------	--	---------------------------------------

3 Risks and Mitigations

3.1 Hardware Failure

There is always a risk of a hardware failure in the middle of development. At any stage, this could significantly affect progress and prevent deadlines from being reached. While the likelihood of this occurring is minimal, the impact would be significant if not catastrophic, especially further down the timeline. To prevent this, I will be keeping my code and research on GitLab and other cloud-based platforms. I will also be keeping a version stored to a physical disk drive. This also does not limit development to my local machine and can be developed elsewhere. While there still is a possible risk for reoccurrence or copies being lost, having multiple copies should allow for one to always be available.

3.2 Poor Estimation of Tasks

Due to the length of the project, there is minimal room for errors in the period spent on each task. If specific features were to take longer to the point where it overruns into other deadlines, this could create a knock-on effect that derails the whole plan. To mitigate this, splitting each task into its components and using a multi-factor estimation method, and then later using the historic data to evaluate the best course of action should reduce the likelihood of this occurring. Any features that have a dependency will also be addressed early on, allowing for concurrent development to exist if required.

3.3 Balance between the Report and Code

There is a significant risk that the app development can overtake the completion of the report. This could make documentation more difficult to revisit as past actions could be forgotten. To mitigate this, I will be maintaining my diary and workbook that will have consistent dated updates. I will also be dedicating a timetabled slot every week for specifically the report, so it does not get overshadowed completely by development. I will be honest in my reflections and put into perspective the progress made.

3.4 End-User Safety

Users can be tracked via their GPS and account details can be shared amongst peers. This could introduce malicious actors who could greatly harm users. While the likelihood is minimal, the impact could be detrimental to users' safety and well-being. All sensitive features such as GPS and public information will be highlighted to users and can be turned off at any time. Data will be stored securely, with personal data likely being stored locally to the user, and users will have limited access to sharing information. This will not completely negate possible risk but makes it harder for malicious actors to possibly gain access.

3.5 Sensitive Data Collection

Many of the user would be sharing account and sensitive personal data with the platform, and if this were to be compromised could be extremely dangerous for users. Similarly to the end-user's safety, while this would be unlikely, the impact would be significant. To mitigate this, details of users will be encrypted and stored securely, passwords will not be stored in the clear and strict access controls will be implemented. Any templates will be stored separately from any user databases to avoid accidental cross over.

3.6 Scope Creep

Scope creep can lead to a difficulty in understanding when to stop adding features, which can result in missed deadlines or underdeveloped features. To prevent this, I will be making a clear plan of what I want to achieve and adhering to a strict schedule. After the first term, I will re-evaluate the progress made and use that evidence to plan the second term. Features will also be given a priority allowing for opportunities for non-essential features to be removed in favour of not compromising the overall quality.